

Tablewise Operator's Manual v1.0

How to Operate Tablewise Without Having to Remember Anything

Founder's Edition

Who this is for: Suvir, future contributors, contractors, family members helping run Tablewise, and early employees.

What this is: a plain-English operating playbook. It tells you **exactly what to do** in common situations. It is **not** an architecture document — if you want the engineering detail, read `docs/HANDOFF.md` and `docs/ARCHITECTURE.md`. This manual assumes you want to get something done without holding the whole system in your head.

The promise of this manual: if you can read and follow a checklist, you can operate Tablewise safely. You do not need to remember anything. You only need to find the right section.

Table of contents

1. Cover / purpose
 2. Golden rule: follow the checklist
 3. The cast: who does what
 4. Big-picture workflow
 5. Daily startup checklist
 6. "I have a new idea" workflow
 7. "Claude finished work" workflow
 8. "I need a review" workflow
 9. "I am on my phone" workflow
 10. Session closeout checklist
 11. Commit / push / deploy decision guide
 12. Emergency procedures
 13. Common commands
 14. Things we never do
 15. Glossary
 16. Quick reference appendix
-

1. Cover / purpose

Tablewise is a household meal-intelligence app (it used to be called "Metabolic Meal OS" / "Meal OS"). This manual is about **operating the development of Tablewise** — running the AI assistants, keeping work organized, and not breaking anything — not about using the app itself.

The whole system is built around one simple idea:

Work lives in files, not in your head or in a chat window.

Every task, every handoff, every rule is written down in the `/ai` folder. When you sit down (or pick up your phone), you read the relevant file and follow it. That is the entire operating model.

Where everything lives:

```
Project folder: /Volumes/Mac Mini - Extended/Projects/metabolic-meal-os

README.md ..... front door, points to everything
/ai/ ..... the AI operating workflow (read these first)
  AGENT_RULES.md ..... the rules every AI assistant must follow
  CLI_ROLES.md ..... which tool does what + what's installed
  CURRENT_TASK.md ..... the ONE job being worked on right now
  TASK_PACKET_TEMPLATE.md the form used to define a job
  CLOSEOUT_CHECKLIST.md .. the end-of-job checklist
  SESSION_HANDOFF.md ..... the logbook of who did what, handed to whom
  PROMPT_LIBRARY.md ..... copy-paste prompts for each AI tool
  MOBILE_WORKFLOW.md ..... how to operate from an iPhone
  AGENTOPS_V2_PROPOSAL.md future plan (proposed, not active)
  BUILDER_OS_SALVAGE_PLAN.md lessons from an old prototype
/docs/ ..... engineering + operating docs (incl. this manual)
/scripts/ ..... helper scripts you run by name
```

2. Golden rule: follow the checklist

★ THE GOLDEN RULE

When in doubt, find the checklist and follow it top to bottom. Do not improvise. Do not skip steps. Do not trust memory. If there is no checklist for your situation, stop and ask Suvir.

Everything else in this manual is just checklists for specific situations. The skill you need is not "remember how Tablewise works" — it is "find the right checklist and follow it honestly."

✅ DO THIS

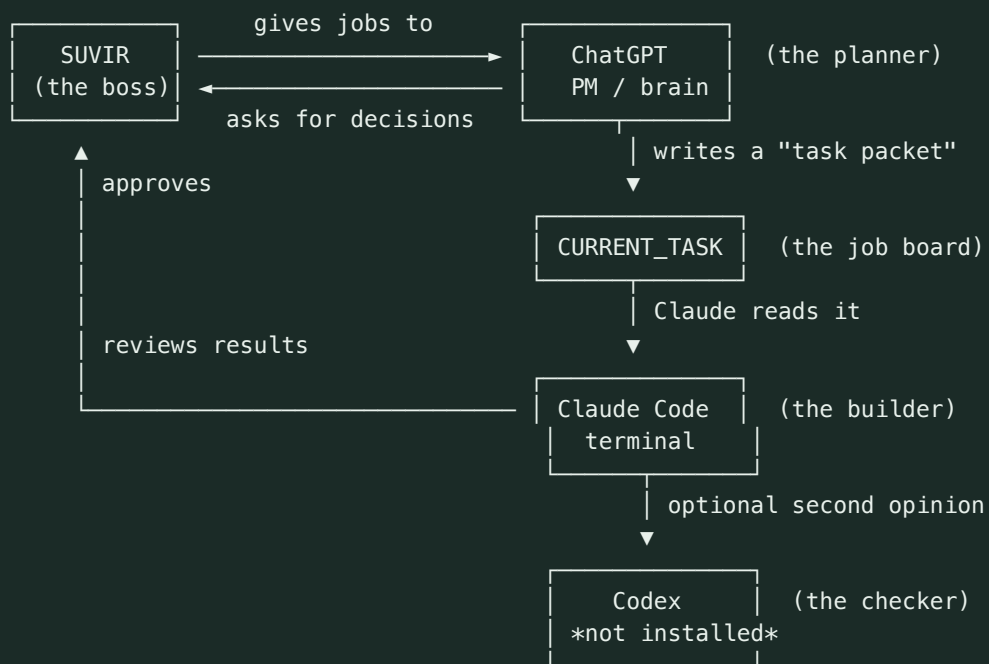
- Read the file before acting.
- Run the steps in order.
- Write down what actually happened (including failures).

❌ DO NOT DO THIS

- Guess.
- Skip the validation gate "just this once."
- Say a step passed when you did not run it.

3. The cast: who does what

Think of Tablewise development as a small team. Each member has one job. Staying in lane is what keeps them from stepping on each other.



Who	Role in plain English	Where they live
Suvir	The boss. Decides what matters, approves commits/deploys, makes the calls only a human should make.	Anywhere
ChatGPT	The planner / project manager. Turns a fuzzy idea into a clear written job ("task packet"). Does not touch the code.	Web browser
Claude Code	The builder. Does the actual coding, runs the checks, updates the docs. The main worker.	Terminal on the Mac Mini
Codex	The checker / second opinion. Reviews work or makes one small fix. Not installed yet — treat as optional.	Terminal (when added)
Mac Mini	The machine that actually runs everything. Always on, holds the project.	The Mac Mini
Termius + tmux	Your remote control from a phone. Termius is the app that connects; tmux keeps the work alive if your phone disconnects.	iPhone → Mac Mini

Important: Today only **Claude Code** and the project's built-in commands are installed. Codex, Gemini, aider, and other tools are *candidates* — they may not exist on the machine. Never assume a tool is installed; the script in §13 tells you the truth. Never install one to "unblock" yourself.

4. Big-picture workflow

This is the whole life of a piece of work, start to finish:

```
IDEA
|
▼
[1] ChatGPT turns the idea into a TASK PACKET
    |
    (one small, clear slice of work)
    ▼
[2] The packet is placed in ai/CURRENT_TASK.md
    |
    (now there is exactly ONE active job)
    ▼
[3] Claude Code reads it and builds the slice
    |
    ▼
[4] Claude runs the VALIDATION GATE
    |
    lint → typecheck → test → build
    |
    └─ ✗ fails → fix, run again (do not proceed on red)
    |
    ▼ ✓ passes
[5] (Optional) Codex or Claude reviews the change
    |
    ▼
[6] Claude fills the CLOSEOUT CHECKLIST + updates HANDOFF
    |
    ▼
[7] SUVIR decides: commit? push? deploy? stop? ← see §11
    |
    ▼
DONE – handoff written in ai/SESSION_HANDOFF.md
```

Every arrow is a checklist in this manual. You never have to invent the next step — you just look it up.

5. Daily startup checklist

Do this every time you begin a Tablewise session, on desktop or phone.

✓ STARTUP

1. Get to the project.

```
cd "/Volumes/Mac Mini - Extended/Projects/metabolic-meal-os"
```

1. See the lay of the land.

`bash scripts/agent-status.sh` This tells you: which tools are installed, whether the checks are available, the active task, and (on a phone) your tmux session.

1. **Read the active job.** Open `ai/CURRENT_TASK.md`. Who owns it? What's the status? If it says "**No active task**", there is nothing in flight.

1. **Read the last handoff.** Open the top entry of `ai/SESSION_HANDOFF.md` to see what happened last and what's next.

1. Check for uncommitted work. `git status --short`

- Empty output = clean, nothing pending.
- Lines listed = there is work in the tree; read it before changing anything.

1. **Now decide which workflow you're in** (§6–§9) and go to that section.

✓ DO THIS

- Always run `agent-status.sh` first. It is read-only and safe.

🚫 DO NOT DO THIS

- Start editing before reading `CURRENT_TASK.md`. You might stomp on work another session left in progress.

6. "I have a new idea" workflow

You thought of something Tablewise should do. Here is how to turn it into safe, organized work.

```
graph TD
    A[Your idea] --> B[Talk it through with ChatGPT (planner)]
    B --> C["Break this into the smallest safe slice."]
    C --> D[ChatGPT writes a TASK PACKET (using TASK_PACKET_TEMPLATE.md)]
    D --> E[You paste/save that packet into ai/CURRENT_TASK.md]
    E --> F[Hand it to Claude Code to build ($7 covers what comes back)]
```

✓ NEW IDEA

1. Open ChatGPT. Describe the idea in plain words.
2. Ask: **"Break this into the smallest safe vertical slice and fill out the Tablewise task packet."** (The template is `ai/TASK_PACKET_TEMPLATE.md`.)
 1. Make sure the packet answers: **"What's the goal? Does it change how the app behaves? Which files? What does 'done' look like?"**
 1. Save the finished packet into `ai/CURRENT_TASK.md`. There should only ever be **one** active task there.
 1. Tell Claude Code: **"Read `ai/CURRENT_TASK.md` and the rules, then build it."** (Use the Claude prompt in `ai/PROMPT_LIBRARY.md`.)

✓ DO THIS

- Keep slices small. One screen, one fix, one feature at a time.
- Write down what is **out of scope** so it doesn't sneak in.

⊘ DO NOT DO THIS

- Hand Claude a giant vague idea ("make the planner better"). It will guess.
- Run two tasks at once. One active task, one owner.

7. "Claude finished work" workflow

Claude Code says it's done. Before you believe it, walk this checklist.

✓ CLAUDE FINISHED

1. **Did the validation gate pass?** Ask for the results, or run:

`bash scripts/agent-closeout.sh` You want to see: **lint pass, typecheck pass, test pass, build pass.**

1. **Look at what changed.** `git status --short` then `git diff`

Read it. Does it match what the task asked for? Anything surprising?

1. **Was the closeout filled in?** Check `ai/CLOSEOUT_CHECKLIST.md` — real results, not blanks.

1. **Was the handoff written?** Check the top of `ai/SESSION_HANDOFF.md`.

2. **Decide what to do with it** → go to §11 (commit / push / deploy / stop).

```
Claude says "done"
  |
  ▼
Gate green? — no → send it back. Not done.
  | yes
  ▼
Diff matches the task? — no → ask why. Maybe send back.
  | yes
  ▼
Closeout + handoff written? — no → ask Claude to finish paperwork.
  | yes
  ▼
Go to §11 (commit/push/deploy/stop)
```

⊘ DO NOT DO THIS

- Accept "all tests pass" without seeing the numbers. Ask for "369/369"-style evidence.
- Commit a diff you have not actually looked at.

8. "I need a review" workflow

You want a second pair of eyes on a change before trusting it.

✓ REVIEW

1. Make sure the change is in the working tree and the gate is green (\$?).
2. If **Codex is installed** (check with `agent-status.sh`): hand it the diff in **audit mode** — read-only, list problems, don't edit. Use the Codex audit prompt in `ai/PROMPT_LIBRARY.md`.
1. If **Codex is NOT installed** (the normal case today): ask Claude Code to do a self-review pass, or run the built-in `/code-review` in a Claude session.
 1. Collect the findings. Decide: fix now, log as follow-up, or accept as-is.
 2. Record the decision in `ai/CURRENT_TASK.md` notes.

✓ DO THIS

- Prefer the reviewer to be a **different** tool than the one that wrote the code.

⊘ DO NOT DO THIS

- Install Codex just to get a review. If it's not there, use Claude's review instead.
- Let the reviewer start editing without first taking ownership via a handoff.

9. "I am on my phone" workflow

You're away from the Mac Mini and want to operate from your iPhone using **Termius** (the SSH app) and **tmux** (keeps work alive across drops). The full guide is `ai/MOBILE_WORKFLOW.md`; this is the operating version.

🔑 THE PHONE GOLDEN RULE

Always work inside tmux. If your phone locks or loses signal, tmux keeps everything running on the Mac Mini. Without it, a dropped connection kills your work.

```
iPhone (Termius) —SSH→ Mac Mini → tmux session "tablewise"
                                |
lock phone / lose signal → you DETACH (work keeps running)
                                |
reopen Termius → you ATTACH → right back where you were
```

☑ PHONE SESSION

1. Open **Termius** → tap the Mac Mini host → open an SSH session.
2. **Reconnect to your work** (attach if it exists, else start it):

```
tmux attach -t tablewise || tmux new -s tablewise
```

1. Go to the project if needed:

```
cd "/Volumes/Mac Mini - Extended/Projects/metabolic-meal-os"
```

1. **Orient:** `bash scripts/agent-status.sh` (now shows your tmux session).
 2. Do safe work (see box below). Start long jobs, then **detach** and lock the phone — come back later for the result.
1. **Detach (safe, keeps running):** press `Ctrl-b`, release, then press `d`.

☑ SAFE FROM A PHONE

- `bash scripts/agent-status.sh` — orientation
- `bash scripts/agent-closeout.sh` — run the checks (detach, come back)
- `git status`, `git diff`, `git log --oneline -n 20` — look around
- Reading files; small, well-scoped Claude Code edits you can read on screen

🚫 NEVER FROM A PHONE

- Commit or push
- Database migrations or deploys
- Installing anything / editing secrets (`.env*`)
- Big multi-file changes you can't fully read on a small screen
- Save those for a desktop where you can review the whole diff.

If your phone disconnects: don't panic. Your work is still running in tmux on the Mac Mini. Just reopen Termius and run the attach command in step 2.

10. Session closeout checklist

Before you walk away, leave the project clean and the next person (maybe future you) informed. This mirrors `ai/CLOSEOUT_CHECKLIST.md` and the end-of-session procedure in `docs/HANDOFF.md`.

✓ CLOSEOUT

1. **Run the gate** (only if you changed scripts or app code):

`bash scripts/agent-closeout.sh` — record real results.

1. **Fill the closeout file:** `ai/CLOSEOUT_CHECKLIST.md` with actual pass/fail.

2. **Write the handoff:** add a top entry to `ai/SESSION_HANDOFF.md` —

what changed, what's verified, what's next, who owns the tree now.

1. **Update the job board:** set the status in `ai/CURRENT_TASK.md`

(Complete / Blocked) or replace it with the next packet.

1. **Update docs if app behavior or routes/env changed** (`docs/HANDOFF.md` ,

`docs/ROADMAP.md` , `docs/SESSION_LOG.md` , etc.).

1. **List the files you changed** and write a one-line commit recommendation.

2. **Leave the tree clean or clearly noted.** Run `git status --short` so the next person sees exactly what's pending.

⊘ DO NOT DO THIS

- Leave without writing a handoff. The next session won't know what you did.
- Claim the gate passed if you skipped it — say "skipped, docs only" instead.

11. Commit / push / deploy decision guide

This is the most important decision in the manual. **Committing, pushing, and deploying are separate steps, and each one needs a reason.** Default to doing nothing until Suvir says so.

```

Work is done and the gate is green.
    |
    v
Did Suvir (or the task) ask to COMMIT?
    |
    |— no —> STOP. Leave it in the working tree. You're done.
    |
    |— yes
    |
    v
Are you on the main branch?
    |
    |— yes —> make a branch first:
    |           git checkout -b <type>/<short-name>
    |
    |— no —>
    |
    v
Commit (scoped, clear message). — then —>
    |
    v
Did Suvir ask to PUSH?
    |
    |— no —> STOP. Commit stays local.
    |
    |— yes
    |
    v
Push the branch. — then —>
    |
    v
Did Suvir ask to DEPLOY?
    |
    |— no —> STOP. Pushed, not deployed.
    |
    |— yes
    |
    v
Deploy is a DESKTOP, reviewed action. Never from a phone.

```

Action	When it's OK	Never
Commit	Suvir asked; gate is green; you've read the diff.	On a whim; on red; on <code>main</code> without a branch.
Push	Suvir asked; commit reviewed.	"To back it up." Pushing is publishing.
Deploy	Suvir asked; on a desktop; change reviewed.	From a phone; without review; to "just try it."

✅ DO THIS

- Treat "do nothing yet" as a valid, safe outcome.
- Branch off `main` before committing if you're on `main`.

❌ DO NOT DO THIS

- Auto-commit because the work looks finished. **Default is: do not commit.**
- Force-push, reset `--hard`, or delete branches without explicit instruction.

12. Emergency procedures

Calm, specific steps for when something goes wrong. None of these are catastrophic if you follow the steps.

"I think I broke something / the checks are failing"

1. Don't commit. Don't push.
2. See what's changed: `git status --short` and `git diff`.
3. If the change is small and wrong, you can throw away **un-committed** edits to a single file: `git checkout -- <path/to/file>` (this discards your changes to that file — only do it if you're sure).

1. Run the gate again: `bash scripts/agent-closeout.sh`.
2. Still broken? Write what happened in `ai/SESSION_HANDOFF.md` and ask Suvir.

"My phone disconnected mid-task"

1. Your work is still running in tmux on the Mac Mini. Nothing is lost.
2. Reopen Termius → SSH to the Mac Mini.
3. `tmux attach -t tablewise` — you're back.
4. If `tmux ls` shows the session is gone, the job finished or the machine restarted; check `git status --short` to see the result.

"Two sessions edited at once / I'm confused who owns the work"

1. Stop editing.
2. Read `ai/CURRENT_TASK.md` (who's the owner?) and the top of `ai/SESSION_HANDOFF.md`.
1. Whoever is **not** named owner backs off. Ownership passes only via a written handoff entry.

"A secret/key might have been exposed"

1. Do **not** commit or push anything containing it.
2. Do not paste it into ChatGPT, a browser, or any external tool.
3. Tell Suvir immediately — keys may need rotating (see `docs/HANDOFF.md`).

"I don't know what to do"

1. That's fine. Run `bash scripts/agent-status.sh`.
2. Read `ai/CURRENT_TASK.md` and the latest handoff.
3. Find the matching section in this manual (§5–§11).
4. Still stuck? Stop and ask Suvir. Stopping is always safe.

🚫 IN ANY EMERGENCY, NEVER

- Force-push or `git reset --hard` to "clean things up."
- Delete files or branches to make an error go away.
- Run database migrations or deploys to "reset" the situation.

13. Common commands

You can operate Tablewise with this short list. Copy-paste; don't memorize.

```
# Go to the project
cd "/Volumes/Mac Mini - Extended/Projects/metabolic-meal-os"

# See what's installed + active task + tmux state (READ-ONLY, always safe)
bash scripts/agent-status.sh

# Run all the checks (lint, typecheck, test, build). Safe to run anytime.
bash scripts/agent-closeout.sh
bash scripts/agent-closeout.sh --db    # also run the read-only DB check

# Look around (all read-only)
git status --short
git diff
git log --oneline -n 20

# The individual checks, if you want them one at a time
npm run lint
npm run typecheck
npm test
npm run build
npm run db:check    # read-only database check

# --- Phone (tmux) ---
tmux attach -t tablewise || tmux new -s tablewise    # reconnect to work
tmux ls                                              # list running sessions
# Detach (keep it running): press Ctrl-b, then d
```

Everything above is safe to run. The dangerous actions (commit, push, deploy, migrate, install) are **not** in this list on purpose — those need a decision (§11), not a reflex.

14. Things we never do

🚫 THE NEVER LIST

- **Never commit or push unless Suvir asks.** Default is leave-it-in-the-tree.
- **Never deploy or run a database migration from a phone.**
- **Never install dependencies or new CLIs** to unblock yourself. Candidate tools that aren't installed are simply not options this session.
- **Never print, paste, or commit secrets** or `.env*` files. Not into ChatGPT, not anywhere.
- **Never change how the app behaves** during a docs/tooling task.
- **Never run two active tasks at once.** One job, one owner.
- **Never claim a check passed that you didn't run.** Say "skipped" honestly.
- **Never force-push, reset --hard, or delete branches** without explicit instruction.
- **Never revive Builder-os as a foundation** (see the lesson below).

The Builder-os lesson

Builder-os was an **earlier prototype** — a web app that tried to manage AI development through a browser GUI and a server. It worked, but it was the wrong shape: it needed a backend, stored everything in fragile browser storage, and added a server to maintain.

The lesson: Tablewise AgentOps deliberately does the same job with plain files and small scripts instead. We **salvage good ideas** from Builder-os (like its role prompts and its "stop after this slice" discipline) but we **do not revive the app**. The full reasoning is in `ai/BUILDER_OS_SALVAGE_PLAN.md`.

Plain version: *old prototype = idea donor, not a thing to bring back to life.*

15. Glossary

Term	Plain meaning
AgentOps	Our system for running AI assistants without them stepping on each other. Lives in <code>/ai</code> .
Task packet	A filled-out form describing one small job. Made from <code>ai/TASK_PACKET_TEMPLATE.md</code> .
CURRENT_TASK	The one job being worked on right now (<code>ai/CURRENT_TASK.md</code>).
The gate / validation gate	The four checks every change must pass: lint, typecheck, test, build.
lint	Checks the code's style/quality.
typecheck	Checks the code's types line up (catches a class of bugs).
test	Runs the automated tests.
build	Confirms the app can be assembled for release.
Closeout	The end-of-job checklist (<code>ai/CLOSEOUT_CHECKLIST.md</code>).
Handoff	The logbook passing work between sessions/tools (<code>ai/SESSION_HANDOFF.md</code>).
Owner	The one person/tool allowed to edit the project right now.
Working tree	Your uncommitted changes — work that exists but isn't saved into history yet.
Commit	Saving a change into the project's history (local).
Push	Sending committed changes to the shared/remote copy (publishing).
Deploy	Putting changes live for real users.
tmux	Software that keeps your terminal work alive even if your phone disconnects.
Termius	The iPhone app used to connect to the Mac Mini.
Attach / Detach	Reconnect to / step away from a tmux session (work keeps running when detached).
CLI	A "command-line tool" — something you run by typing its name.
Candidate tool	A tool we <i>might</i> use but that isn't installed (e.g. Codex, Gemini).
ChatGPT	The planner. Turns ideas into task packets. Doesn't touch code.
Claude Code	The builder. Does the coding in the terminal.
Codex	The optional checker/second-opinion tool (not installed yet).
Builder-os	An old prototype. Idea donor only — never revived.

16. Quick reference appendix

The one-screen version. (Also available as its own file: `docs/TABLEWISE_OPERATOR_QUICK_REFERENCE.md`, and a phone card: `docs/TABLEWISE_MOBILE_CARD.md`.)

Start every session:

```
cd "/Volumes/Mac Mini - Extended/Projects/metabolic-meal-os"
bash scripts/agent-status.sh
# then read ai/CURRENT_TASK.md and the top of ai/SESSION_HANDOFF.md
```

Run the checks:

```
bash scripts/agent-closeout.sh
```

The four-question decision (after work is done & green):

```
Commit?  only if Suvir asked → branch off main first
Push?    only if Suvir asked
Deploy?  only if Suvir asked → desktop only, never phone
Else:    STOP. Leave it in the tree.
```

Phone:

```
tmux attach -t tablewise || tmux new -s tablewise  # reconnect
# detach (safe): Ctrl-b then d
```

Never: commit/push/deploy/migrate/install without being asked · print secrets · run two tasks · revive Builder-os.

When lost: run `agent-status.sh`, read `CURRENT_TASK.md`, find the matching section above, or ask Suvir. Stopping is always safe.

End of Tablewise Operator's Manual v1.0 — Founder's Edition. *This is an operating playbook. For engineering detail see `docs/HANDOFF.md`.*